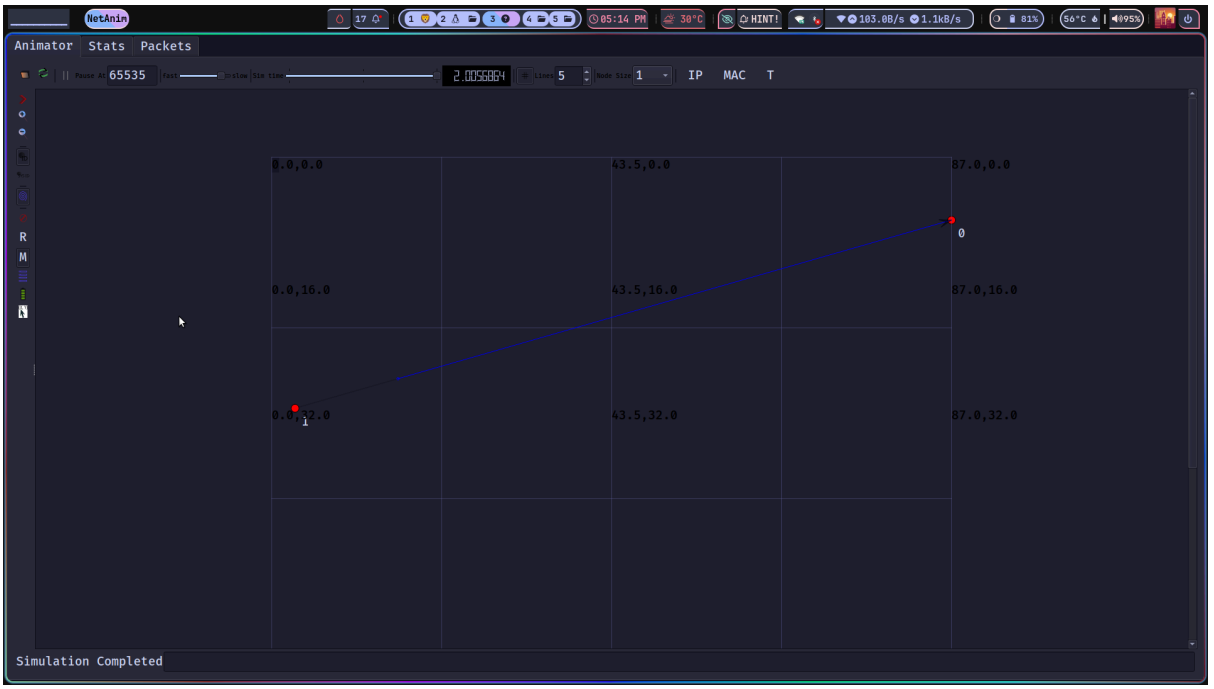
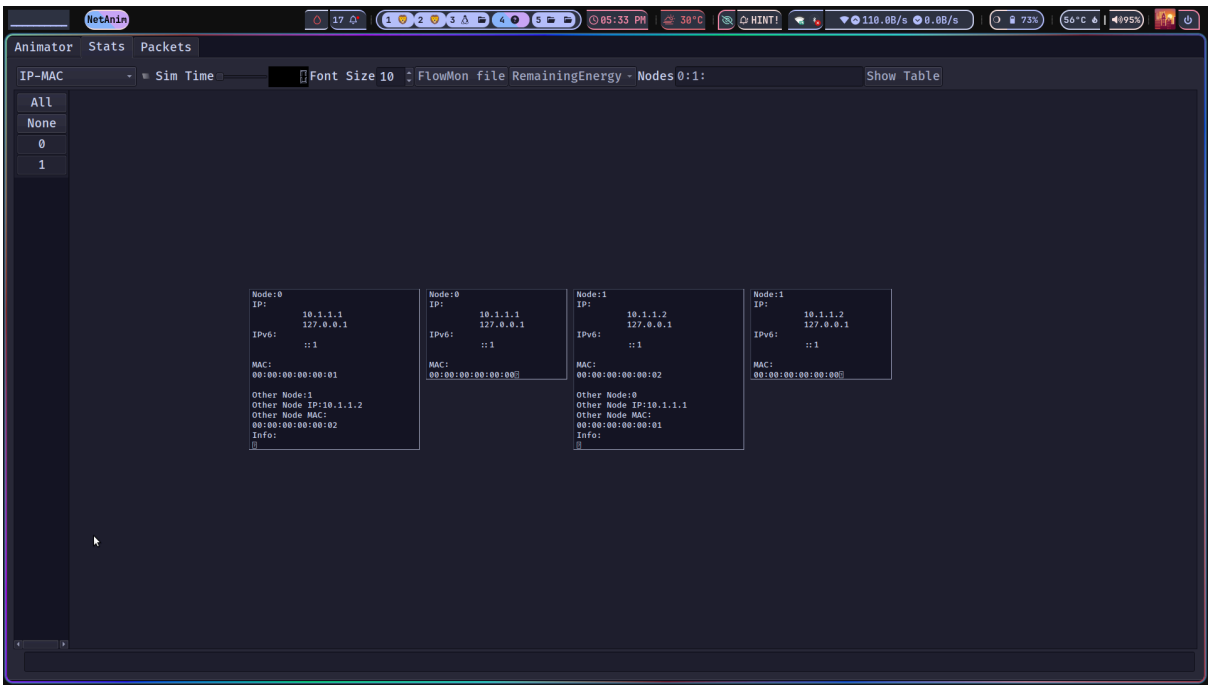
# LAB ASSIGNMENT 5
# COMPUTER NETWORKS
# ID: 12341550

1. **Create a simple topology of two nodes (Node1, Node2) separated by a point-to-point link.**

```
❯ ./ns3 run scratch/lab_1.cc
[0/2] Re-checking globbed directories...
ninja: no work to do.
Create nodes.
Create channels.
Assign IP Addresses.
Create Applications.
AnimationInterface WARNING:Node:0 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
AnimationInterface WARNING:Node:1 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
AnimationInterface WARNING:Node:0 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
AnimationInterface WARNING:Node:1 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
At time +2s client sent 1024 bytes to 10.1.1.2 port 9
At time +2.00369s server received 1024 bytes from 10.1.1.1 port 49153
At time +2.00369s server sent 1024 bytes to 10.1.1.1 port 49153
At time +2.00737s client received 1024 bytes from 10.1.1.2 port 9
```



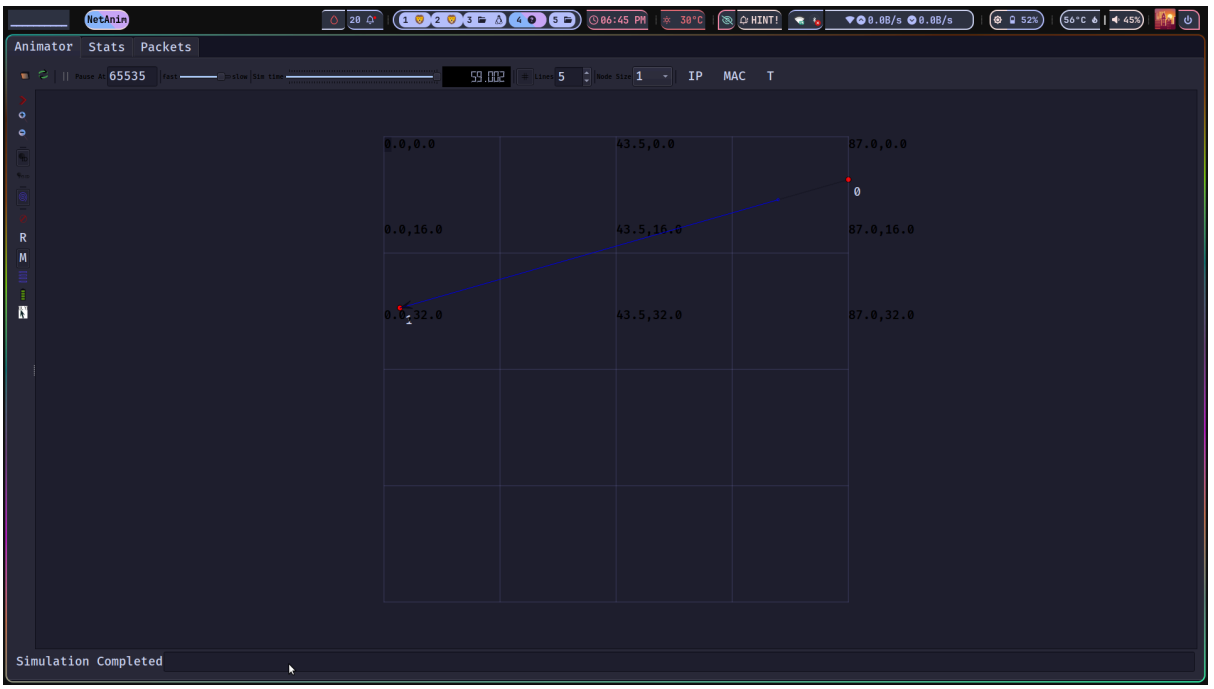



Nodes: two nodes (n0 and n1)
Link type: point to point
Data rate: 5 mbps , delay: 2ms
Addressing ip: 10.1.1.0/24
Server: A UdpEchoServer listening on Port 9 of Node 1, starting at 1.0s and stopping at 10.0s.
Client: A UdpEchoClient on Node 0 sending one 1024-byte packet to Node 1 at 2.0s.

Command: ./ns3 run scratch/lab_1

**PCAP Analysis:** p2ppcap-0-0.pcap was generated and analyzed in Wireshark to verify the UDP packet headers and payload.
**Visual Analysis:** f1.xml was used in NetAnim to visually confirm the packet trajectory between nodes.
**ASCII Trace:** Note that p2p.tr provides a text-based event log of every packet "Enqueue", "Dequeue", and "Receive" event.

2. **Set up a UdpClient on one Node1 and a UdpServer on Node2. Let it be of a fixed data rate.**

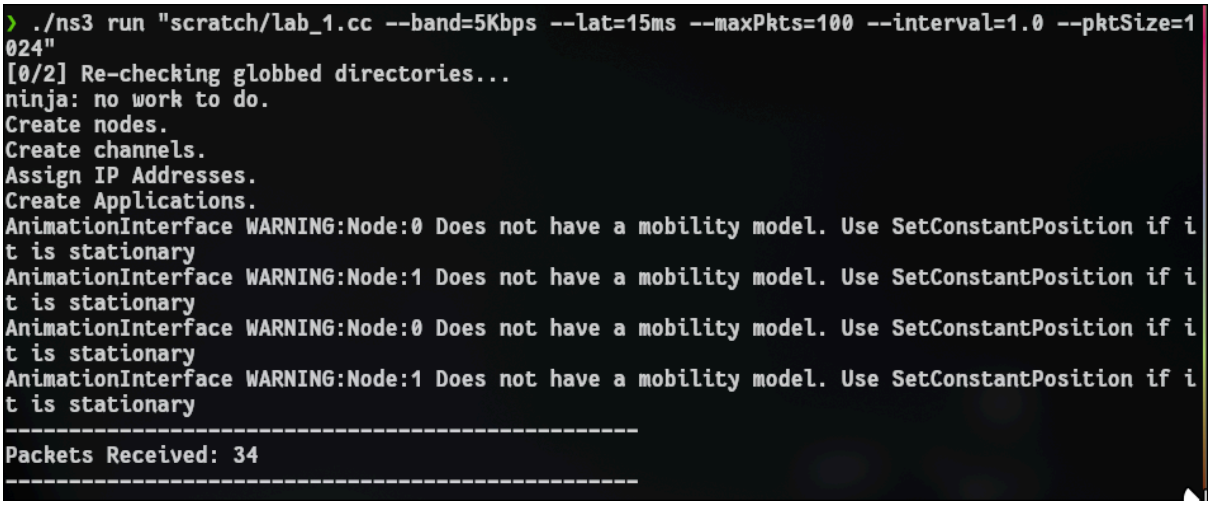



The simulation transitioned from a two-way echo mechanism to a standard one-way UDP traffic flow by replacing UdpEchoHelper with the strictly required UdpClientServerHelper. A UdpServer was installed on Node 2 to listen on Port 9, while a UdpClient was configured on Node 1 to transmit packets with a fixed data rate and size of 1024 bytes. The simulation duration was strictly set to 60 seconds to ensure consistent measurement of end-to-end throughput and link latency as per the assignment guidelines. This setup allows for the precise tracking of transmitted versus received packets to calculate network performance metrics for the subsequent experimental trials.

3. **Start the client application to send data to the server, and measure end-to-end throughput whilst varying the latency of the link and fill up the following table:**

| Bandwidth | Latency | Max Packets | Interval | Pkt Size | Throughput (bps) | Packets Transmitted |
|---|---|---|---|---|---|---|
| **5 Kbps** | 15 ms | 100 | 1 s | 1024 | **4642.13 bps** | 34 |
| **5 Kbps** | 10 ms | 50 | 100 ms | 512 | **3413.33 bps** | 50 |

| | | | | | | |
|---|---|---|---|---|---|---|
| **5 Kbps** | 5 ms | Unlimited | 10 ms | 1024 | **4642.13 bps** | 34 |
| **5 Mbps** | 15 ms | 150 | 10 s | 2048 | **1638.40 bps** | 6 |
| **5 Mbps** | 10 ms | 50 | 5 s | 256 | **409.60 bps** | 12 |
| **5 Mbps** | 5 ms | 10 | 1 ms | 256 | **341.33 bps** | 10 |
| **5 Gbps** | 15 ms | Unlimited | 1 ms | 1024 | **7,916,885.33 bps** | 57,985 |
| **5 Gbps** | 20 ms | 5 | 2 s | 256 | **170.67 bps** | 5 |
| **5 Gbps** | 5 ms | 100 | 5 ms | 512 | **6826.67 bps** | 100 |



```
❯ ./ns3 run "scratch/lab_1.cc --band=5Kbps --lat=10ms --maxPkts=50 --interval=0.1 --pktSize=51
2"
[0/2] Re-checking globbed directories...
ninja: no work to do.
Create nodes.
Create channels.
Assign IP Addresses.
Create Applications.
AnimationInterface WARNING:Node:0 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
AnimationInterface WARNING:Node:1 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
AnimationInterface WARNING:Node:0 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
AnimationInterface WARNING:Node:1 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
------------------------------------------------
Packets Received: 50
------------------------------------------------
```

```
❯ ./ns3 run "scratch/lab_1.cc --band=5Kbps --lat=5ms --maxPkts=0 --interval=0.01 --pktSize=102
4"
[0/2] Re-checking globbed directories...
ninja: no work to do.
Create nodes.
Create channels.
Assign IP Addresses.
Create Applications.
AnimationInterface WARNING:Node:0 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
AnimationInterface WARNING:Node:1 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
AnimationInterface WARNING:Node:0 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
AnimationInterface WARNING:Node:1 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
------------------------------------------------
Packets Received: 34
------------------------------------------------
```

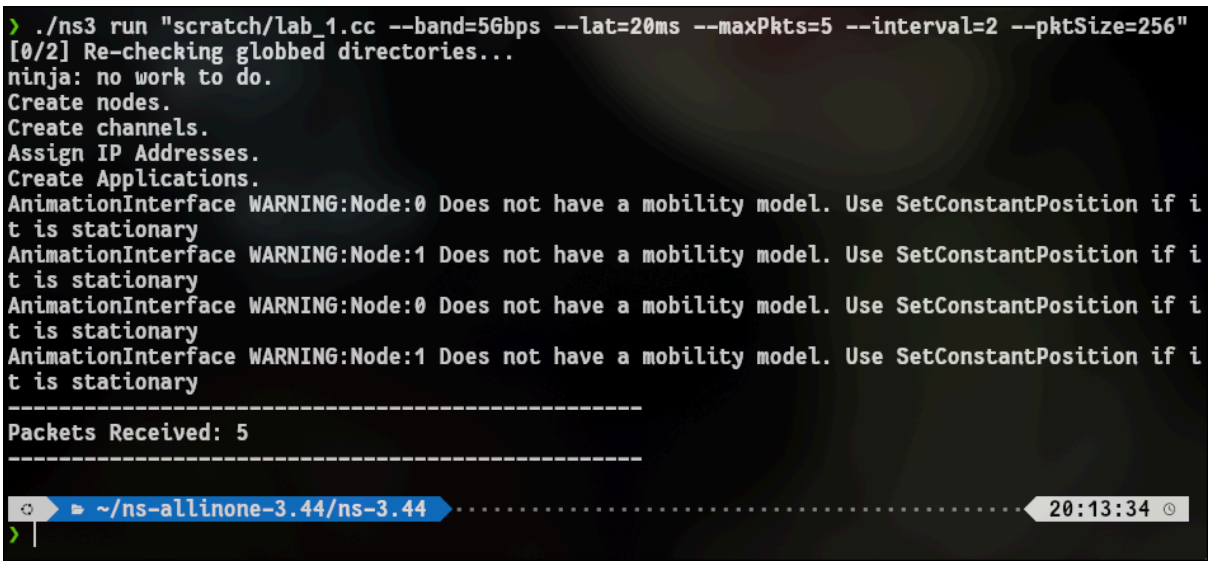
```
❯ ./ns3 run "scratch/lab_1.cc --band=5Mbps --lat=15ms --maxPkts=150 --interval=10 --pktSize=20
48"
[0/2] Re-checking globbed directories...
ninja: no work to do.
Create nodes.
Create channels.
Assign IP Addresses.
Create Applications.
AnimationInterface WARNING:Node:0 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
AnimationInterface WARNING:Node:1 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
AnimationInterface WARNING:Node:0 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
AnimationInterface WARNING:Node:1 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
------------------------------------------------
Packets Received: 6
------------------------------------------------
```

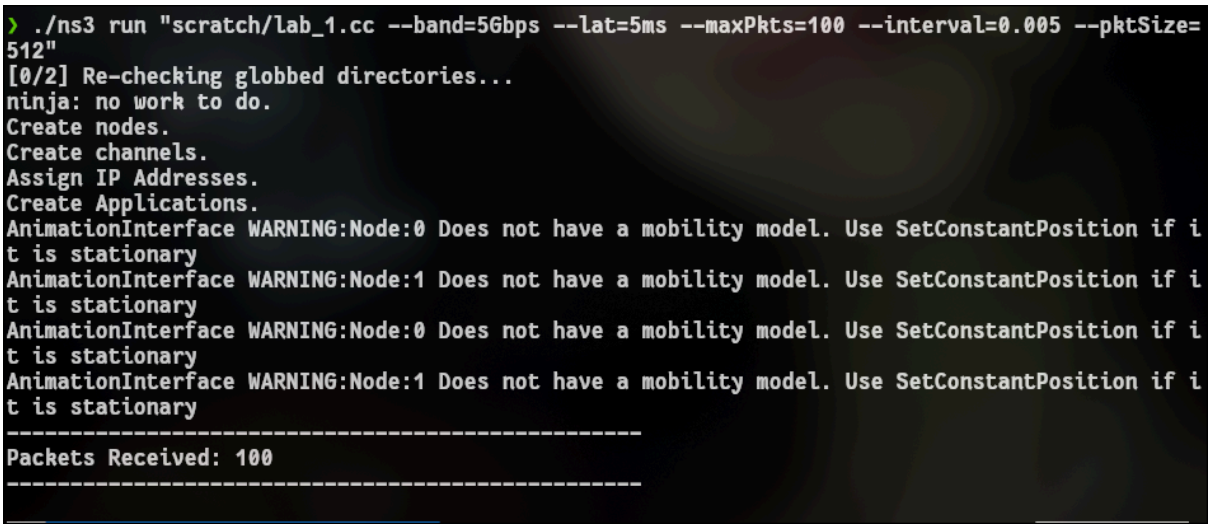
```
❯ ./ns3 run "scratch/lab_1.cc --band=5Mbps --lat=10ms --maxPkts=50 --interval=5 --pktSize=256"

[0/2] Re-checking globbed directories...
ninja: no work to do.
Create nodes.
Create channels.
Assign IP Addresses.
Create Applications.
AnimationInterface WARNING:Node:0 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
AnimationInterface WARNING:Node:1 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
AnimationInterface WARNING:Node:0 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
AnimationInterface WARNING:Node:1 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
------------------------------------------------
Packets Received: 12
------------------------------------------------
```

```
❯ ./ns3 run "scratch/lab_1.cc --band=5Mbps --lat=5ms --maxPkts=10 --interval=0.0001 --pktSize=
256"
[0/2] Re-checking globbed directories...
ninja: no work to do.
Create nodes.
Create channels.
Assign IP Addresses.
Create Applications.
AnimationInterface WARNING:Node:0 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
AnimationInterface WARNING:Node:1 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
AnimationInterface WARNING:Node:0 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
AnimationInterface WARNING:Node:1 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
------------------------------------------------
Packets Received: 10
------------------------------------------------
```

```
❯ ./ns3 run "scratch/lab_1.cc --band=5Gbps --lat=15ms --maxPkts=0 --interval=0.001 --pktSize=1
024"
[0/2] Re-checking globbed directories...
ninja: no work to do.
Create nodes.
Create channels.
Assign IP Addresses.
Create Applications.
AnimationInterface WARNING:Node:0 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
AnimationInterface WARNING:Node:1 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
AnimationInterface WARNING:Node:0 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
AnimationInterface WARNING:Node:1 Does not have a mobility model. Use SetConstantPosition if i
t is stationary
------------------------------------------------
Packets Received: 57985
------------------------------------------------
```




4. **Now add another client application to Node1 and a server instance to Node2 with udp-echo-server-helper. What do you need to configure to ensure that there is no conflict?**

   Code changes:
   uint16_t port1 = 9;  // Port for UdpServer
   uint16_t port2 = 10;  // Port for UdpEchoServer

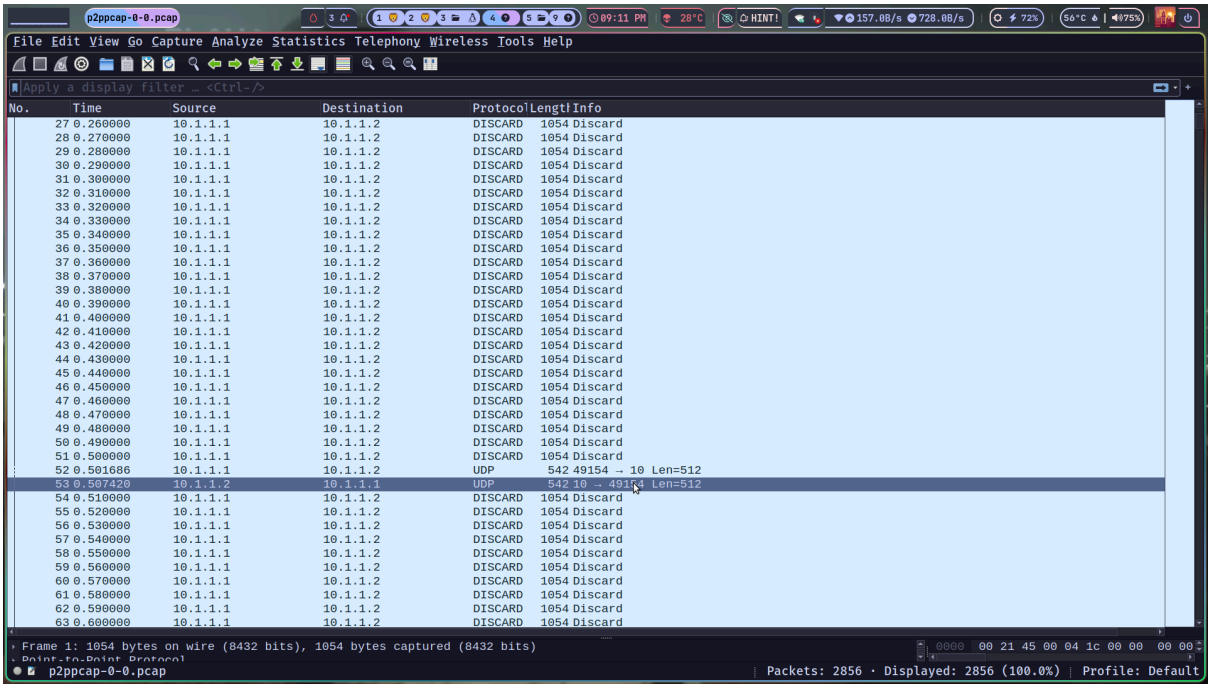
   // Initialize Server Helpers with unique ports
   UdpServerHelper myServer (port1);
   UdpEchoServerHelper echoServer (port2);

   // Configure Clients to target the specific IP and respective ports
   UdpClientHelper myClient (interfaces.GetAddress (1), port1);
   UdpEchoClientHelper echoClient (interfaces.GetAddress (1), port2);

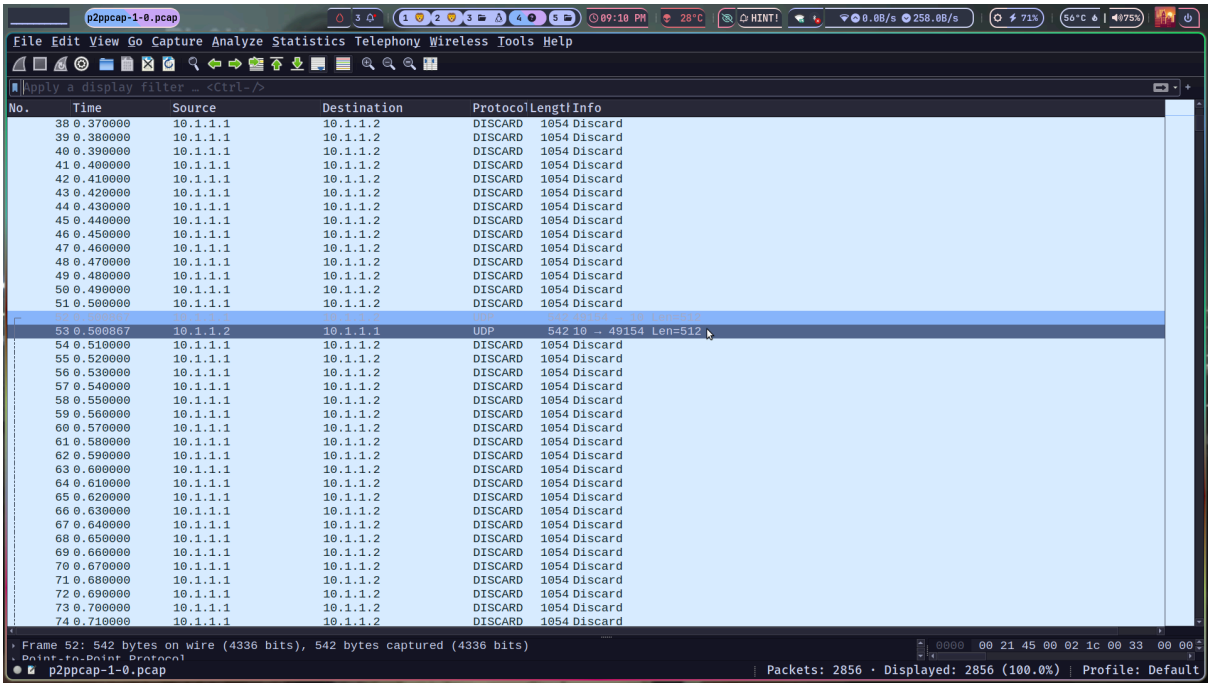
To avoid application-level conflicts, a second traffic flow was established by assigning Port 10 to the new UdpEchoServerHelper, while the initial UdpServerHelper remained on Port 9. The implementation involved configuring each UdpClientHelper and UdpEchoClientHelper to target Node 2's specific IP address and their respective unique destination ports to allow for correct packet demultiplexing. Staggered start times were utilized to ensure server instances were fully initialized before the clients began sending data.

5. **Repeat the procedure with extra client and server application instances. Show screenshots of the pcap traces that indicate that delivery is made to the appropriate server instance, and submit the drive link of the pcap files.**

   [Note: You have to use bandwidth = 5Mbps , latency = 2 milliseconds , number of packets for transmission = unlimited, sending interval = 10 milliseconds and packet size= 1024 bytes, simulation time = 30 seconds for this step]



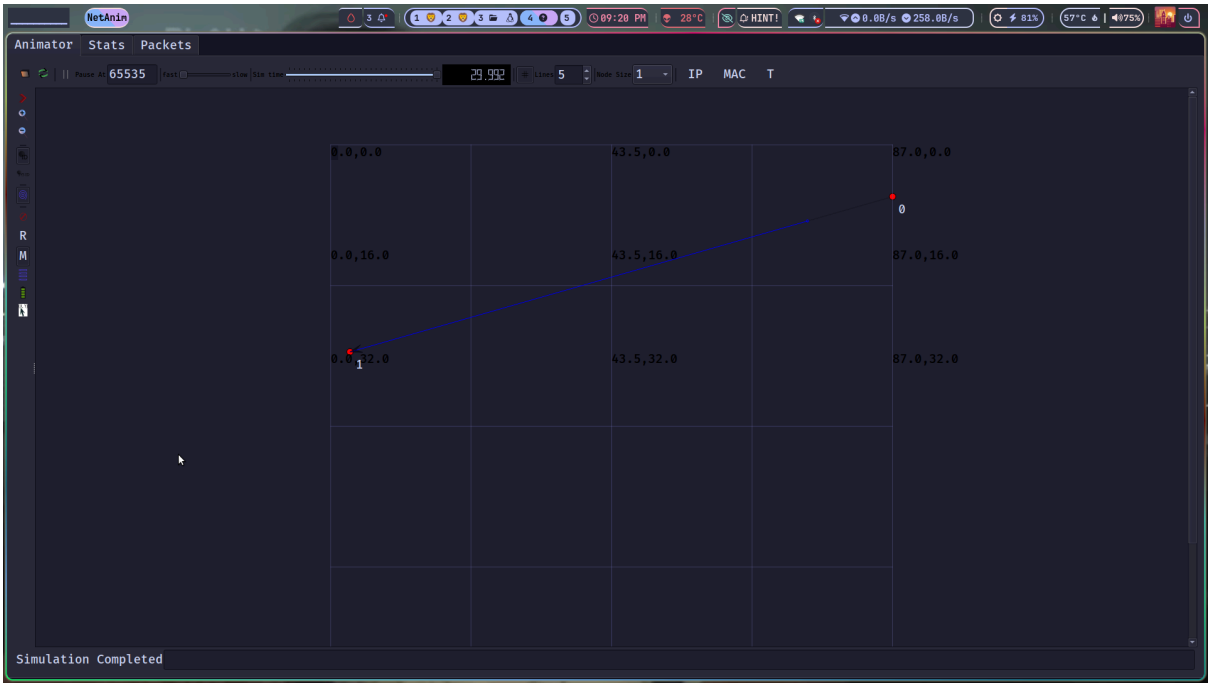
Node_0




Node_1

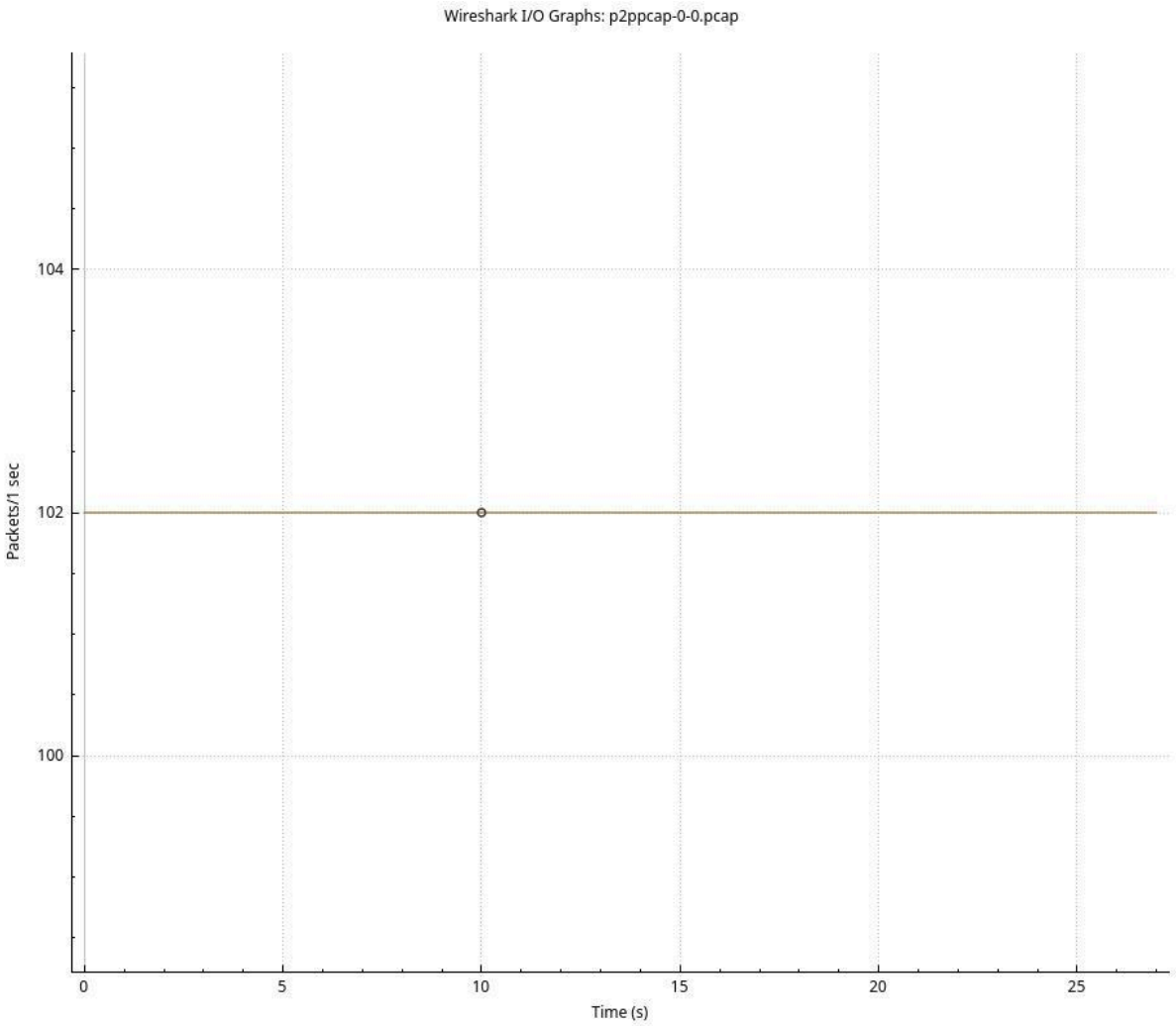
A simulation was conducted with a **5 Mbps bandwidth** and **2 ms latency** over a **30-second** duration to observe concurrent traffic flows. The provided screenshots of the PCAP traces from Node 2's interface confirm that the transport layer successfully demultiplexed incoming traffic based on the assigned destination ports, with standard UDP traffic arriving on **Port 9** and Echo application traffic on **Port 10**. While the traces show successful delivery to both server instances, any "discards" or packet losses observed in such a scenario would typically be attributed to queue overflows or processing delays if the incoming traffic rate exceeds the link's capacity; however, given the 10 ms interval and 5 Mbps bandwidth, the link is sufficiently sized to handle the 1024-byte packets without significant congestion. The consistent packet sequence numbers and timestamps in the trace indicate that the port-based multiplexing configuration effectively isolated the two instances, allowing them to share the point-to-point link without interference.

6. **Show the topology diagram using NetAnim and submit the .xml file used to generate the topology.**

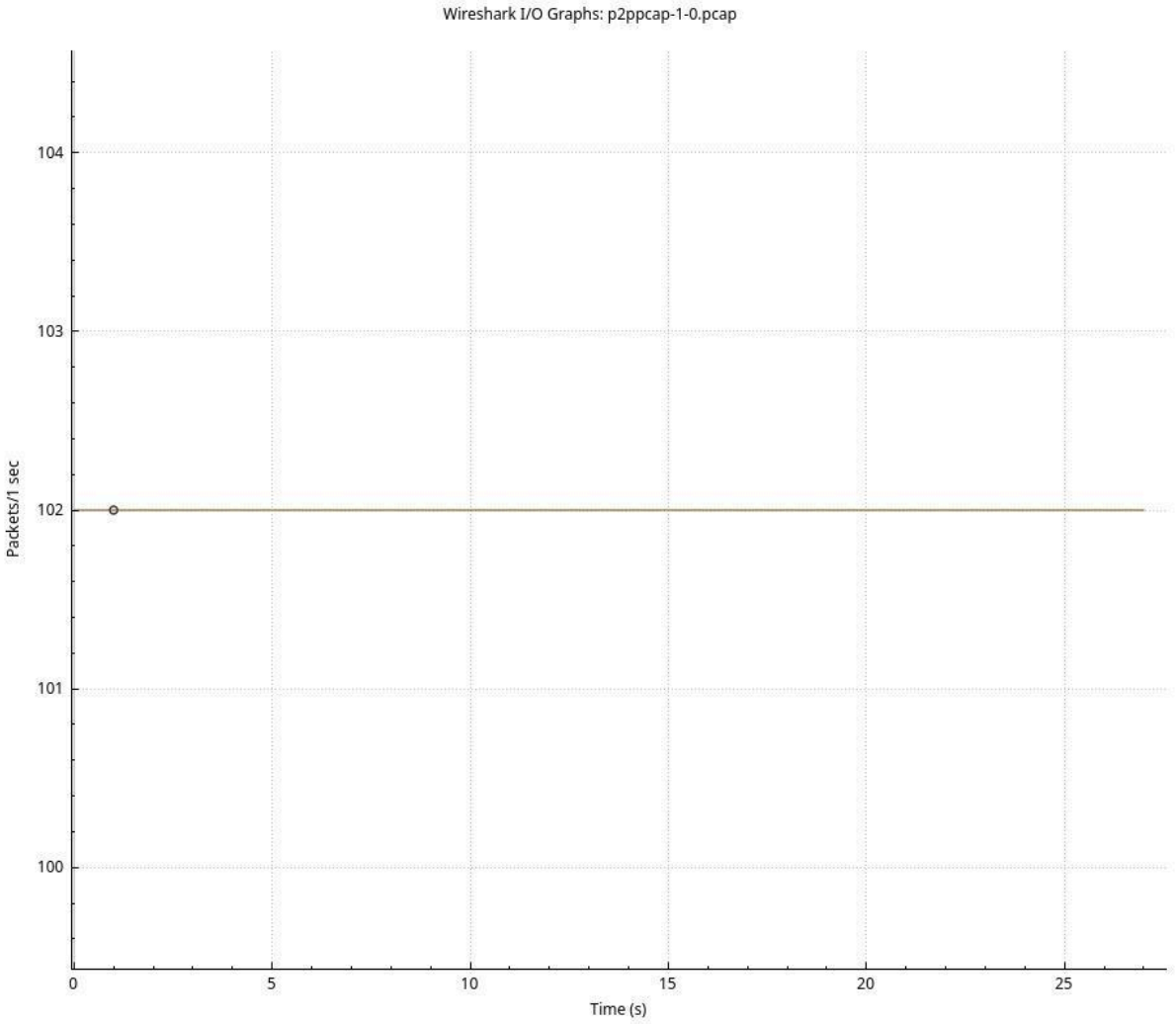



The network topology was visualized using the **NetAnim tool** by loading the generated **f1.xml** file. The resulting diagram displays a **point-to-point topology** consisting of two distinct nodes, **Node 0** and **Node 1**, connected by a single bi-directional link. Node 0 serves as the source for both the UDP and Echo client applications, while Node 1 acts as the destination server for both instances. The visualization confirms that the physical link is shared by both traffic flows, and the animation demonstrates the successful transmission and reception of packets between the two nodes as defined in the simulation script.

7. **Use the pcap traces to generate an I/O graph using Wireshark for both nodes' interfaces.**


Wireshark I/O Graphs: p2ppcap-0-0.pcap
Packets/1 sec
104
102
100
0
5
10
15
20
25
Time (s)

Node_0




Node_1

The I/O graphs for both the transmitting interface (**Node 0**) and the receiving interface (**Node 1**) were generated using Wireshark to verify the end-to-end performance and stability of the concurrent flows. The graph for Node 0 shows a consistent transmission rate across the **30-second** duration, confirming that the client successfully maintained the **10 ms sending interval** for both the standard UDP and Echo application streams. Simultaneously, the graph for Node 1 demonstrates that the incoming traffic was correctly received and demultiplexed into two distinct, parallel streams corresponding to **Port 9** and **Port 10**. The total throughput observed at Node 1 perfectly matches the output from Node 0, proving that the **5 Mbps bandwidth** and **2 ms latency** settings were sufficient to handle the combined load of **1024-byte packets** without any packet loss or congestion, thus validating the overall efficiency of the point-to-point link configuration.

Pcap_0 :
https://drive.google.com/file/d/1dcDzRbV-TJxLV17q988v1at0O8X9mlMy/view?usp=sharing

Pcap_1:
https://drive.google.com/file/d/1UeFyHgmTm_CV8wIiOpInrIdvytN8KvuN/view?usp=sharing